FUNDAMENTALS OF DATA SCIENCE

ASSIGNMENT

Name: Pranesh G
Section-C

Reference Raw Table (Before Preprocessing Starts)

Student ID	Name	Age	Gender	Marks	Attendance	City
101	Ravi	20.0	M	85	90	chennai
102	Priya	19.0	F	92	93	chennai
103	Arjun	NaN	M	78	88	madurai
104	meena	20.0	F	430	91	chennai
105	Ravi	20.0	M	85	90	Chennai
106	Kiran	20.0	Male	NaN	NaN	Coimbatore

Step 1: Data profiling

Structure Discovery: Inspecting Schema and Data types

Python code:

```
df.info()
```

Function name: DataFrame.info()

Syntax breakdown: df.info() analyze memory usage, non-null counts, and dtypes

What Data Profiling Reveals Here: Age, Attendance lack full non-null counts.

Console output:

2 Age 5 non-null float64 <---- (1 missing)

4 Marks 5 non-null float64 <---- (1-missing)

5 Attendance 5 non-null float64 <---- (1-missing)

2. Content Discovery (Part A): Checking Missing values (NULL)

Target Rows and columns: Row and (Age), Row 5 (Marks, Attendance)

Python code:

```
Print (df.isnull().sum())
```

Function Name: DataFrame.isnull().sum()

3. Content discovery (Part B): Value Distribution and outlier detection

Target Rows and columns: Row 5 (Gender), Row 3 (Marks).

Python code:

```
Print (df['Gender'].value_counts())
Print (df['Marks'].describe())
```

Function name: Series value_counts() and DataFrame.describe()

Conside output

Male 1 ←---(Inconsistent label)

Max 450.0 ←---(Domain Error: max >100)

4. Relationship Discovery: checting for Duplicate prefixes and Icey Integrity Target rows and Columns: check all rows for redundancy.

Python code:

Print(df.duplicated(subset = ['Name', 'Age', 'Gender', 'City']).sum())

Function Name: DataFrame.duplicated()

what Data profiling Reveals: 1 duplicate found (Row 4)

Step-2: Data cleansing (Fixing structural Errors and Formatting)

1. Standardizing Inconsistent categorical labels in Grenger Target Row and column: Rows under Gender.

Python code:

df['Gender'] = df['Gender'].replace({'male':'M'})

Function name: Replace()

2. Removing the Duplicate Student profile

Target Row and Column: Row 4 (Ravi)

Python code:

df = df.drop_duplicates(subset=['Name', 'Age', 'Gender', 'City'], keep='first')

Function name: drop duplicates.

Step 3: Constraint Validation (Domain Bounds)

Target Row and column: Row 3 under Marks (Value: 450)

Python Code

df.loc[df['Marks']>100, 'Marks'] = np.nan

Function name: df.loc[]

Step 4: Missing Data Imputation (Nan Handling)

Target Rows and Columns: Age (row 2), Marks (row 3, 5), Attendance (row 5).

Calculation: Median Age (20.0), Mean marks (85.0), Mean attendance (91.0)

Python Code:

df['Age'] = df['Age'].fillna(df['Age'].median())

df['Marks'] = df['Marks'].fillna(df['Marks'].mean())

df['Attendance'] = df['Attendance'].mean()

Function used : fillna(), median(), mean()

Syntax is correct

Step-5 Data transformation

Target columns: Mark, Attendance, Age, City.

Python code:

```
df['marks_per_Attendance'] = df['marks'] / df['Attendance']
df['Age_group'] = pd.cut(df['Age'], bins=[0, 20, 100],
                         labels = ['Teen', 'Adult'])
region_map = {'Chennai': 'North TN', 'Coimbatore': 'West TN',
              'Madurai': 'South TN'}
df['Region'] = df['City'].map(region_map)
from sklearn.preprocessing import MinMaxScaler
df['Norm_marks'] = MinMaxScaler().fit_transform
                                  (df['Marks'])
```

Functions used : /, pd.cut(), map(), MinMaxScaler()

Step 6: Data Reduction:

Target Columns to drop: Student_ID, Name

Python code:

```
df_reduced = df.drop(Column=['Student_ID',
                             'Name'])
```

Function Name: DataFrame.drop().

Step 7 : Data Enrichment

Target Rows and External Table: Join cal_index onto City

Python code:

```
City_cal_table = pd.DataFrame({'city':'Chennai',
                               'Coimbatore', 'Madurai'],'col_index'
                               [1.2,1.0,0.9]})
df_enriched = pd.merge(df_reduced, city_cal_table
                       ,on='City', how='left')
```